Performance Analysis of Sparse Matrix-Vector Multiplication (SpMV)

Michael Bernasconi
Student ID: 267681
michael.bernasconi@studenti.unitn.it
https://github.com/Michael-Bernasconi/Sparse-Matrix-Vector-Multiplication
Department of Information Engineering and Computer Science
University of Trento

***Abstract*—Sparse Matrix-Vector multiplication (SpMV) is a fundamental cornerstone of numerical computing; its efficiency on GPUs is historically limited by irregular memory accesses and load imbalance. This study analyzes the performance of custom implementations (COO, CSR, CSR-Vector) by comparing them with the cuSPARSE library on NVIDIA Ampere architecture (A30). A novel evaluation is proposed that correlates matrix topology with hardware efficiency: the results demonstrate that the CSR-Vector approach, through warp-level parallel reduction, mitigates computational divergence of irregular matrices, achieving a peak of 95.73 GFLOPS and saturating up to 390.83 GB/s of effective bandwidth. A distinctive contribution of this work is the analysis of Time To Solution (TTS), which reveals that I/O parsing costs and PCIe transfers dominate the program lifecycle.**

***Index Terms*—SpMV, CUDA, Sparse Matrix Vector Multiplication, Load Balancing, NVIDIA Ampere, Memory Bandwidth, Time To Solution (TTS), Throughput, Performance Evaluation.**

I. Introduction

The Sparse Matrix-Vector Multiplication (SpMV) operation, defined as $y = Ax$, constitutes a critical computational kernel for High Performance Computing (HPC) [1].

Due to its low arithmetic intensity, quantifiable as $2 \times nnz$ total operations, SpMV is an inherently memory-bound operation. Its efficiency is further hindered by the irregular nature of memory accesses, which strongly depend on the sparsity structure of the matrix.

This study analyzes the impact of storage formats and parallelization strategies on NVIDIA Ampere architecture, comparing custom CUDA implementations (COO, CSR, and CSR-Vector) with the NVIDIA cuSPARSE library.

The investigation evaluates performance through key metrics: kernel execution time, throughput (GFLOPS), effective bandwidth (BW), and overall *Time To Solution* (TTS).

This study correlates structural characteristics of the matrix with hardware performance, offering an experimental guide for selecting the optimal format and kernel in order to mitigate computational and system bottlenecks.

II. Methodology

This section describes the test environment, the state-of-the-art context for SpMV optimizations, the matrix formats examined, and the rigorous methodology adopted for performance measurement and validation.

A. Hardware and Software Environment

Computations were performed on the UniTN cluster (node edu01) on the edu-short partition. Hardware specifications are summarized in Table I

TABLE I: Hardware Environment Specifications

Feature	Host (CPU)	Device (GPU)
Model	Intel Xeon Silver 4309Y	NVIDIA A30
Architecture	Ice Lake (x86_64)	Ampere (Compute Cap. 8.0)
Core / SM	16 Core / 32 Thread (2 Sock.)	56 SM (3584 CUDA Core)
Clock Frequency	2.80 GHz (Base) / 3.60 GHz	1.44 GHz (Boost)
FP32 Performance	1.84 TFLOPS	10.3 TFLOPS
Memory Bandwidth	102.4 GB/s	933.1 GB/s (HBM2)
Total Global Memory	N/A	24 GB
Shared Memory	N/A	48-100 KiB / SM
Warp Size	N/A	32

B. State of the Art in SpMV Optimization

SpMV is a critical bottleneck in HPC due to bandwidth saturation and irregular memory access patterns. Although the CSR format was originally designed to minimize memory footprint in sequential contexts, GPU architectures require strategies to mitigate warp divergence and improve memory coalescing [2]. Modern approaches, such as cuSPARSE and warp-based kernels, use cooperative strategies to handle row variance [3].

C. CPU Baseline and GPU Implementations

CPU Baseline: The reference computation on the CPU was handled through a parallel implementation based on OpenMP. This computational phase is not intended as a performance benchmark, but is used for validation and correctness checking of results subsequently computed in parallel on the GPU.

In GPU computation, SpMV performance critically depends on the interaction between the sparse matrix memory layout and the adopted execution model. To analyze this dual aspect (data structure and computational logic), our investigation focuses on the following approaches:

1) COO Format and Base Kernel: The Coordinate (COO) format explicitly stores the sparse matrix through three arrays of size equal to the number of non-zero elements (NNZ): row

Algorithm 1: SpMV Kernel: COO Base (Atomic-based)

Require: val, row_ind, col_ind (arrays of size nnz), x (dense vector)
Ensure: y (output vector)
$i \leftarrow$ Global thread index
if $i < nnz$ **then**
  $v \leftarrow val[i]$
  $r \leftarrow row\_ind[i]$
  $c \leftarrow col\_ind[i]$
  AtomicAdd($y[r], v \times x[c]$) //Handle race conditions on y
end if

indices, column indices, and values. As described in Algorithm 1, the kernel maps one thread per element.

The Base COO kernel adopts a direct mapping strategy: it launches a single thread for each non-zero element. This approach guarantees excellent load balancing since it is insensitive to row distribution. However, since multiple threads may update the same row concurrently, it requires `atomicAdd` to resolve race conditions, limiting throughput.

2) CSR Format and Base Kernel (Scalar): The Compressed Sparse Row (CSR) format optimizes memory footprint by compressing row indices into a `row_ptr` vector of size $M+1$, following the logic of Algorithm 2. Let M and N denote the number of rows and columns of the sparse matrix A, and nnz the number of its non-zero elements.

Algorithm 2: SpMV Kernel: CSR Scalar

Require: row_ptr (size $M+1$), col_idx, val (size nnz), x (dense vector)
$r \leftarrow$ Assigned row index per thread
if $r < M$ **then**
  $sum \leftarrow 0$
  $row\_start \leftarrow row\_ptr[r]$
  $row\_end \leftarrow row\_ptr[r+1]$
  for j from row_start to $row\_end - 1$ **do**
    $sum \leftarrow sum + val[j] \times x[col\_idx[j]]$
  end for
  $y[r] \leftarrow sum$
end if

The Base CSR kernel maps one thread per row. This eliminates atomic operations but suffers from *warp divergence* and non-coalesced memory accesses when rows have significantly different lengths.

3) Optimized CSR-Vector Kernel: To overcome inefficiencies of Base CSR, the CSR-Vector kernel (Algorithm 3) redefines execution granularity by assigning an entire warp (32 threads) to process a single row. This enables coalesced memory access and uses shared memory for parallel reduction.

Finally, all custom implementations were compared with the cuSPARSE library, which serves as a production-level baseline for NVIDIA Ampere architecture.

Benchmark accuracy is ensured by the following methodological choices:

- *Output Consistency:* To prevent accumulation of incorrect values across iterations (a critical phenomenon in COO format using `atomicAdd`), the destination vector is explicitly reset before each kernel launch.

Algorithm 3: SpMV Kernel: CSR-Vector (Warp-level Parallelism)

Require: row_ptr, col_idx, val, x
$tid \leftarrow$ threadIdx.x //Local index in block
$lane \leftarrow tid \pmod{32}$ //Index within the Warp
$row \leftarrow (\text{blockIdx.x} \times \text{blockDim.x} + tid)/32$
if $row < M$ **then**
  $sum \leftarrow 0$
  $start \leftarrow row\_ptr[row]$
  $end \leftarrow row\_ptr[row+1]$
  for $j = start + lane$ **to** $end - 1$ **step** 32 **do**
    $sum \leftarrow sum + val[j] \times x[col\_idx[j]]$ //Coalesced access
  end for
  $shared\_data[tid] \leftarrow sum$
  WarpSync()
  for $offset \in \{16, 8, 4, 2, 1\}$ **do**
    if $lane < offset$ **then**
      $shared\_data[tid] \leftarrow$
      $shared\_data[tid] + shared\_data[tid + offset]$
    end if
    WarpSync()
  end for
  if $lane = 0$ **then**
    $y[row] \leftarrow shared\_data[tid]$
  end if
end if

- *Variability Mitigation and Measurement:* Pure computation time (*Kernel Time*) is evaluated, ignoring initial I/O (parsing) and memory allocation costs. To obtain stable measurements, each test is divided into two phases:
  - Warm-up (20 iterations): Necessary to stabilize hardware and bring GPU frequencies to steady state.
  - Benchmark (500 iterations): Actual performance measurement.

  The entire procedure is repeated for 5 independent *runs*. The average time recorded during benchmark phases (accompanied by standard deviation to prove stability) becomes the baseline parameter on which all final performance metrics are computed: Kernel Time, Time to Solution (TTS), Throughput (GFLOPS), and Effective Bandwidth (BW).

D. Performance Metrics

Implementations are evaluated using the following metrics and equations:

- Kernel Time (T_{avg}): Recorded using `cudaEvents` to sample exclusively the kernel time window on VRAM, avoiding CPU-related latencies.
- Time To Solution (TTS): Measured using native C libraries (`my_time_lib.h`), capturing the full program lifecycle from startup to validation completion. Includes Matrix Market file parsing costs, `cudaMalloc`, and `cudaMemcpy` overhead.
- Computational Throughput (GFLOPS): The standard assumes 2 float operations (FMA) per non-zero element: $GFLOPS = \frac{2 \times \text{NNZ}}{T_{avg} \times 10^9}$.
- Effective Bandwidth (BW) (GB/s): Being a *memory bound* operation, bandwidth varies with format.
  For CSR: $BW_{CSR} = \frac{4(M+1+\text{nnz})+4(\text{nnz}+N+M)}{T_{avg} \times 10^9}$.

For COO: $BW_{COO} = \frac{4(2\times\text{nnz})+4(\text{nnz}+N+M)}{T_{avg}\times 10^9}$, where factor 4 represents bytes for `int32` and `float32`.

- Cache Statistics (CPU): Profiling of the hierarchical cache system was conducted using Valgrind: Miss D1 (%) $= \left(\frac{\text{Miss D1}}{\text{Total D1 Accesses}}\right) \times 100$. It allows quantifying access efficiency and theoretical bandwidth collapse.

E. Correctness and Validation

Validation compares y_{gpu} against the CPU baseline. To handle floating-point non-associativity, a `FAIL` is triggered only if both thresholds are violated:

- Hybrid Error: $(|y_{gpu} - y_{cpu}| > 10^{-4}) \wedge \left(\frac{|y_{gpu}-y_{cpu}|}{|y_{cpu}|+10^{-7}} > 10^{-3}\right)$

This dual-guard model ensures robustness for values near zero while maintaining a 0.1% relative tolerance, quantifying the numerical drift for every failure

III. Dataset

The experimental evaluation is conducted on a selection of 10 sparse matrices from the *SuiteSparse Matrix Collection*, whose details are reported in Table II. These datasets were chosen to represent a wide variety of application domains and structural characteristics, ranging from perfectly balanced circuit simulations to extremely irregular power-law graphs. Notably, this selection aligns with recent high-performance computing literature, as these matrices are extensively utilized for benchmarking in the work of Chu et al. [2].

TABLE II: Structural characteristics of selected SuiteSparse matrices

Matrix Name	NNZ	Avg NNZ/R	Std. Dev.	Structure
ASIC_680ks	1,693,767	2.48	4.16	Regular
bone010	47,851,783	48.50	7.62	Dense, Symmetric
boyd2	1,500,397	3.22	194.91	Irregular
FullChip	26,621,983	8.91	1,806.80	Extreme Imbalance
Ga41As41H72	18,488,476	68.96	105.39	Dense, Symmetric
ldoor	42,493,817	44.63	14.77	Structured
rajat31	20,316,253	4.33	1.11	Very Regular
Rucci1	7,791,168	3.94	0.34	Rectangular
Si41Ge41H72	15,011,265	80.86	126.97	Dense
webbase-1M	3,105,536	3.11	25.35	Power-law

IV. Results

This section presents experimental data obtained from evaluation of SpMV implementations.

A. Computational Throughput (GFLOPS)

Maximum throughput (Figure 1) was recorded by the `CSR-Vector` kernel on matrix `bone010` with 95.73 GFLOPS. While this value is significantly lower than the theoretical peak of the NVIDIA A30 (10.3 TFLOPS), it is consistent with the *memory-bound* nature of the SpMV kernel. Given the low arithmetic intensity of the operation the performance is strictly capped by the HBM2 bandwidth rather than computing power. Reaching $\sim$96 GFLOPS on a device with 933 GB/s theoretical bandwidth implies an efficiency of over 60% of the achievable peak for this specific intensity, which represents a high level of hardware utilization for sparse kernels.

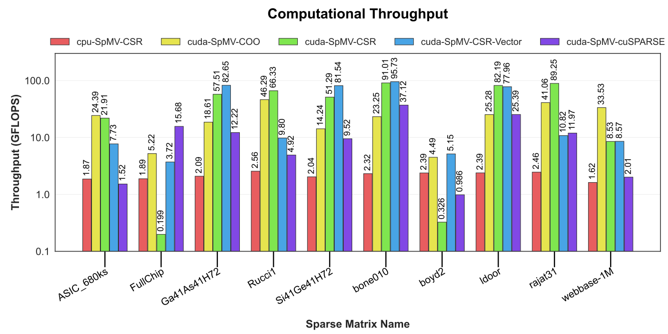

Fig. 1: Computational Throughput (GFLOPS) for tested implementations.

B. Kernel Execution Time

Average times (Figure 2) confirm throughput dynamics. For matrix `FullChip`, execution decreases from 269 ms (`Base CSR`) to 14.1 ms (`CSR-Vector`). The `cuSPARSE` library proves fastest in complex scenarios, recording 3.4 ms on the same matrix.

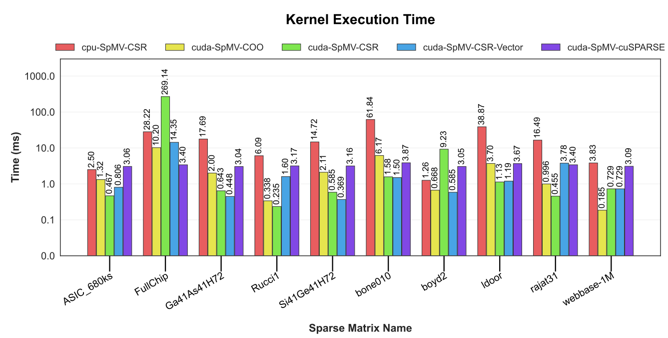

Fig. 2: Average kernel time in seconds with standard deviation.

C. Time To Solution (TTS)

Figure 3 illustrates Total Time To Solution (TTS) of the application. The logarithmic scale graph highlights how inclusion of all preparatory phases tends to level performance differences previously measured on individual kernels. On matrix `ASIC_680ks`, for example, CPU implementation takes about 40.38 s, while the four GPU variants (COO, CSR, CSR-Vector, and cuSPARSE) all converge almost identically between 8.41 s and 8.59 s.

D. Effective Memory Bandwidth

Peak measured value (Figure 4) is 390.83 GB/s (`CSR-Vector` on `bone010`), highlighting a practical limit compared to theoretical bandwidth of NVIDIA A30 (933.1 GB/s). CPU implementations reach a maximum of 13.99 GB/s (`boyd2`).

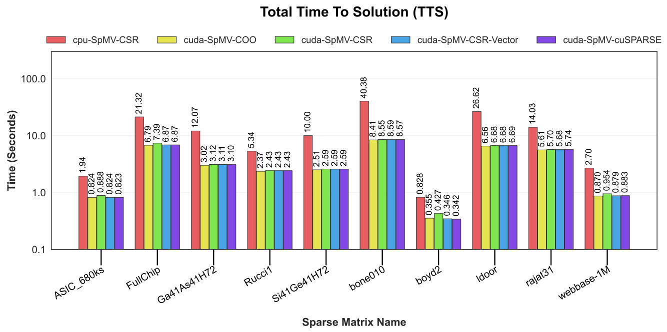

Fig. 3: Total Time To Solution (TTS) in seconds measured on logarithmic scale for different implementations.

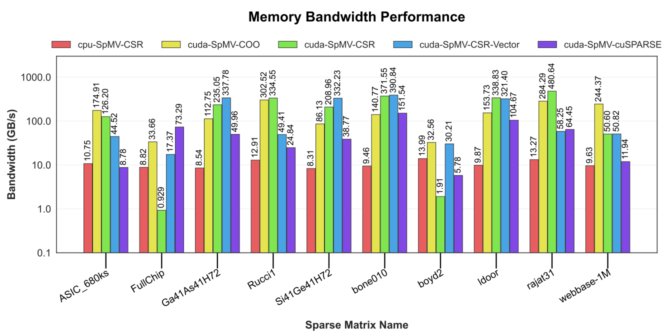

Fig. 4: Effective Memory Bandwidth (GB/s). Theoretical peak of A30 is 933.1 GB/s.

E. Cache Profiling

Analysis via Valgrind detected an extremely low D1 Miss Rate ($< 0.3\%$) for profilable datasets (*ASIC_680ks, boyd2, Rucci1, webbase-1M*), indicating good local data reuse on vectors. Conversely, for large matrices (*bone010, FullChip, ldoor, rajat31*), simulation exceeded the 5-minute limit imposed by the cluster scheduler. This timeout is attributable to high computational overhead of Valgrind when tracing indirect memory accesses typical of SpMV. On datasets with millions of non-zero elements, simulation latency of sparse accesses saturates available time before kernel completion, preventing exhaustive measurement of memory hierarchy.

F. Numerical Validation

The validation procedure returned a `FAIL` state for high-density matrices (*bone010, ldoor*). However, quantitative analysis of the output reveals that the error is marginal: the recorded ϵ_{max} was consistently in the range of 1.2×10^{-3} to 4.5×10^{-3}, only slightly above the 10^{-3} threshold. This confirms that the discrepancy is not due to algorithmic bugs, but to the accumulation of rounding errors in `Float32` parallel reductions (atomic operations and tree reductions), where the non-associativity of floating-point addition becomes measurable on millions of non-zero elements.

V. Discussion

TTS analysis shows that slow disk and PCIe transfers waste GPU speed. Optimization only helps repetitive tasks (like iterative solvers) where loading costs are shared. For "one-shot" tasks, setup time dominates, making GPU speed almost irrelevant. Matrix topology remains the primary performance driver. In irregular matrices, the Base CSR kernel fails due to severe *warp divergence*, where thread idle time scales with row length variance. The CSR-Vector kernel resolves this by assigning a full warp to each row, enabling memory coalescing and efficient parallel reduction.

Despite these optimizations, throughput remains far from the NVIDIA A30 peak. This gap is caused by indirect memory accesses ($x[col\_idx[j]]$): while matrix values are fetched contiguously, input vector elements are accessed via sparsity-dependent patterns, triggering cache misses and fragmented transfers. In this context, cuSPARSE proves superior by employing heuristics that dynamically adapt the execution model to the matrix structure. The fact that peak throughput (95.73 GFLOPS) aligns with bandwidth saturation ($\sim$390 GB/s) confirms that the bottleneck resides in the memory hierarchy rather than CUDA core utilization. Finally, numerical validation shows that while parallelization introduces a slight drift due to floating-point non-associativity, the error remains stable and within acceptable limits for engineering applications.

VI. Conclusion

This study confirms that SpMV efficiency on GPUs is tightly constrained by data topology and memory bandwidth limits.

- Key Results:The CSR-Vector kernel proved essential for handling irregular matrices, overcoming divergence limitations of Base CSR. However, infrastructural costs (I/O and PCIe) represent the real time limitation in single executions.
- Practical Implications: For real-world applications, the use of cuSPARSE is recommended due to its dynamic adaptability. Custom kernel development must carefully consider the trade-off between numerical precision (`Float32`) and speed, especially on high-density datasets.
- Future Work: Further investigations could explore the use of mixed precision and configurable tolerance parameters to mitigate numerical drift observed during validation.

References

[1] J. Gao, B. Liu, W. Ji, and H. Huang, "A Systematic Literature Survey of Sparse Matrix-Vector Multiplication," *arXiv preprint arXiv:2404.06047*, 2024.

[2] G. Chu, Y. He, L. Dong, Z. Ding, D. Chen, H. Bai, x. Wang, and C. Hu, "Efficient Algorithm Design of Optimizing SpMV on GPU," in *Proc. 32nd Int. Symp. High-Perform. Parallel Distrib. Comput. (HPDC '23)*, 2023, pp. 243–256. doi: 10.1145/3588195.3593002.

[3] N. Bell and M. Garland, "Implementing sparse matrix-vector multiplication on throughput-oriented processors," in *Proc. Conf. High Perform. Comput. Netw. Storage Anal. (SC '09)*, 2009, pp. 1–11.

[4] D. Merrill and M. Garland, "Merge-based parallel sparse matrix-vector multiplication," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal. (SC '16)*, 2016, pp. 678–689.